

گزارش جامع راه‌اندازی سخت‌افزاری و مش‌بندی گره‌های ESP۳۲ در پلتفرم RuView (WiFi-DensePose)

امیرعلی یزدانی

۲ شهریور ۱۴۰۵

فهرست مطالب

۳	۱	مقدمه و هدف پروژه
۴	۲	آماده‌سازی محیط توسعه و نصب ESP-IDF
۴	۱۰۲	نصب پیش‌نیازهای سیستمی اوبونتو
۴	۲۰۲	کلون و نصب نسخه ۷۵۰۴
۴	۳	مراحل پیکربندی و فلش فریم‌ورک روی بردهای ESP۳۲-S۳
۴	۱۰۳	لود متغیرهای محیطی و تعیین پورت
۴	۲۰۳	تنظیمات معماری و پیکربندی شبکه (menuconfig)
۴	۳۰۳	پاک‌سازی حافظه قبلی، فلش و راه‌اندازی
۵	۴	راه‌اندازی سرور پردازش حسگری (Sensing Server) و داشبورد تحت وب
۵	۱۰۴	دستور اجرای سرور در حالت سخت‌افزاری
۵	۲۰۴	مشاهده داده‌ها در داشبورد مرورگر
۵	۵	چالش‌های پیش‌آمده و راهکارهای حل آن‌ها
۵	۱۰۵	چالش اول: عدم دریافت پکت‌های UDP توسط سرور (مسدود بودن فایروال)
۶	۲۰۵	چالش دوم: اختلاف زمانی دریافت پکت‌ها در سیستم چندنوده (Timestamp Spread)
۶	۳۰۵	چالش سوم: عبور امواج از دیوار و خطای شناسایی افراد خارج از اتاق
۷	۶	نتیجه‌گیری
۷	۷	روش نوین پیکربندی گره‌ها با استفاده از اسکریپت Provision.py
۷	۱۰۷	مراحل گام‌به‌گام پیکربندی با اسکریپت Provision.py
۷	۱۰۱۰۷	گام اول: راه‌اندازی محیط و اتصال گره
۷	۲۰۱۰۷	گام دوم: تنظیم معماری و پاک‌سازی حافظه فلش
۷	۳۰۱۰۷	گام سوم: فلش فریم‌ورک اصلی
۸	۴۰۱۰۷	گام چهارم: پیکربندی تنظیمات با اسکریپت Provision.py
۸	۲۰۷	پیکربندی گره‌های بعدی در شبکه مش
۸	۱۰۲۰۷	گره شماره ۲ (اسلات ۱)

۸		۲۰۲۰۷	گره شماره ۳ (اسلات ۲)
۹		۳۰۷	مزایای روش Provision.py نسبت به روش سنتی
۹		۴۰۷	نکته مهم در خصوص شناسایی پورت گره‌ها
۹		۵۰۷	اجرای سرور با تنظیمات بهینه
۱۰			آ پوست‌ها و دستورات کمکی شبکه و سخت‌افزار
۱۰		۱.آ	شناسایی پورت سریال گره متصل شده (Port Detection)
۱۰		۲.آ	استخراج آدرس IP کامپیوتر مقصد (Aggregator IP)
۱۰		۳.آ	تغییر IP پس از ری‌استارت و راه‌حل تثبیت آن (Static IP)

۱ مقدمه و هدف پروژه

هدف از این پروژه، پیاده‌سازی و راه‌اندازی سیستم حسگری مبتنی بر سیگنال‌های وای‌فای (WiFi CSI Sensing) با استفاده از پلتفرم متن‌باز RuView و گره‌های سخت‌افزاری ESP32-S3 است. در این فرآیند، استخراج اطلاعات وضعیت کانال (Channel State Information) جهت پایش حضور، تحرک، تنفس و بازسازی فضایی چندگره‌ای (Multi-Static Mesh) به صورت بی‌درنگ انجام گرفته است.

۲ آماده‌سازی محیط توسعه و نصب ESP-IDF

در ابتدای کار، برای سازگاری کامل با فریم‌ورک سورس‌کد RuView و معماری چیپ ESP32-S3، نسخه پایدار **ESP-IDF v5.4** نصب گردید.

۱.۲ نصب پیش‌نیازهای سیستمی اوبونتو

ابتدا پکیج‌های پایه و ابزارهای بیلد در لینوکس اوبونتو نصب شدند:

```
sudo apt-get update
sudo apt-get install -y git wget flex bison gperf python3 python3-pip python3-
  ↳ venv \
  cmake ninja-build ccache libffi-dev libssl-dev dfu-util libusb-1.0-0
```

۲.۲ کlon و نصب نسخه ۵.۴

نسخه ۴.۵ فریم‌ورک به شکل بازگشتی کlon و کامپایلر ابزارهای تارگت esp32s3 نصب شد:

```
mkdir -p ~/esp
cd ~/esp
git clone -b v5.4 --recursive https://github.com/espressif/esp-idf.git
cd esp-idf
./install.sh esp32s3
```

نکته ای که اینجا حتما باید به آن توجه شود این است که توی داکيومنت اصلی به اشتباه نوشته شده ۵.۲ ولی این ورژن مشکل‌های جدی دارد که قابل استفاده نیست.

۳ مراحل پیکربندی و فلش فریم‌ورک روی بردهای S3-ESP32

برای برنامه‌ریزی هر یک از بردها، مراحل زیر به ترتیب اجرا شدند:

۱.۳ لود متغیرهای محیطی و تعیین پورت

قبل از هر کار، متغیرهای محیطی فعال شده و دسترسی کامل به پورت سریال داده شد:
باید پورتی که esp32 به آن متصل شده را در پی‌اریم به طور مثال در این مثال به پورت ۰ttyACM متصل شده است.

```
. $HOME/esp/esp-idf/export.sh
sudo chmod 666 /dev/ttyACM0
cd ~/Desktop/Amirali/WifiSensing/ruview/firmware/esp32-csi-node
```

۲.۳ تنظیمات معماری و پیکربندی شبکه (menuconfig)

ابتدا چیپ تارگت روی esp32s3 تنظیم شده و منوی پیکربندی باز شد:

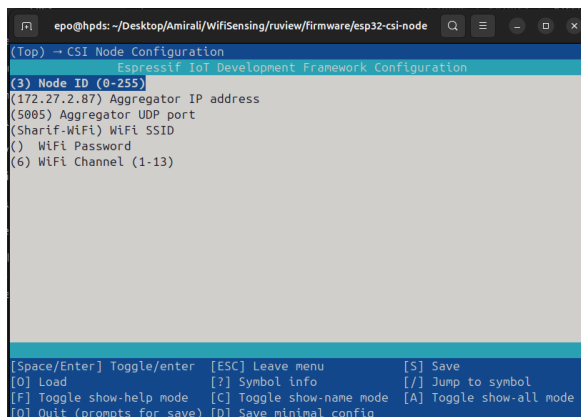
```
idf.py set-target esp32s3
idf.py menuconfig
```

در محیط متنی منو، وارد مسیر زیر شدیم:

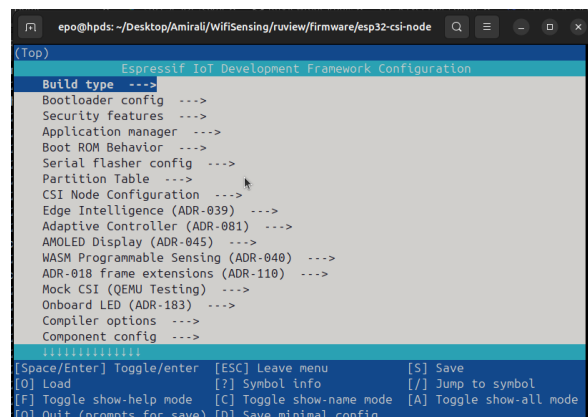
CSI Node Configuration → Component config

پارامترهای زیر با دقت مقداردهی شدند:

- **WiFi SSID**: نام شبکه وای فای محیط (Sharif-WiFi)
- **WiFi Password**: برای شبکه‌های باز خالی گذاشته شد.
- **Aggregator IP**: آدرس آی پی لپ تاپ در شبکه مشترک (به عنوان مثال: 172.27.2.87)
- **Aggregator Port**: مقدار پیش فرض پورت UDP روی ۵۰۰۵
- **Node ID (0-255)**: شناسه یگای هر برد



(ب) منوی پیکربندی پارامترهای گره CSI



(آ) منوی اصلی ESP-IDF

شکل ۱: محیط تنظیمات menuconfig: (الف) منوی اصلی و (ب) منوی تنظیم مشخصات وای فای و شناسه گره

۳.۳ پاک سازی حافظه قبلی، فلش و راه اندازی

برای جلوگیری از تداخل فریم ورک های قبلی، حافظه فلش پاک و کد جدید تزریق شد:

```
idf.py -p /dev/ttyACM0 erase-flash
idf.py -p /dev/ttyACM0 flash monitor
```

نکته مهم سخت افزاری: پس از اتمام فلش، برای رفع باگ لایه فیزیکی Native USB CDC و آغاز بی درنگ اتصال وای فای، دکمه سخت افزاری **RST / EN** روی برد فشرده شد که منجر به سفید شدن ال ای دی وضعیت گردید.

۴ راه‌اندازی سرور پردازش حسگری (Sensing Server) و داشبورد تحت وب

پس از اتمام فلش و روشن شدن گره‌ها، هسته سرور پردازش و ادغام داده‌ها (sensing-server) که با زبان Rust و فریم‌ورک وب Axum توسعه داده شده است، اجرا گردید.

۱.۴ دستور اجرای سرور در حالت سخت‌افزاری

سرور باید در حالت دریافت داده‌های واقعی از ESP32 و به صورت گوش دادن به پورت UDP 5005 در تمام کارت‌های شبکه فعال شود:

```
cd ~/Desktop/Amirali/WifiSensing/ruview/v2
cargo run --release --package wifi-densepose-sensing-server -- \
  --udp-bind 0.0.0.0 \
  --udp-insecure-lan \
  --source esp32
```

۲.۴ مشاهده داده‌ها در داشبورد مرورگر

پس از مشاهده پیام HTTP server listening on 127.0.0.1:8080 در ترمینال، رابط کاربری با باز کردن آدرس زیر در مرورگر (مانند فایرفاکس یا کروم) در دسترس قرار گرفت:

<http://localhost:8080/ui/index.html>

در این صفحه، با رفتن به تب **Sensing**، پارامترهای بلادرنگ سیگنال شامل واریانس حرکت، توان باند تنفسی و فرکانس خام CSI قابل رصد هستند.

یادآوری مهم در خصوص بیلد پروژه: شایان ذکر است که اجرای سریع دستور cargo run در مراحل فوق به این دلیل است که پکیج‌های پروژه پیش‌تر یک‌بار کامپایل و بیلد شده بودند. در صورتی که برای نخستین‌بار یا روی سیستمی جدید این فرآیند انجام شود، ابتدا باید کل پکیج‌های زبان Rust بیلد شوند (که فرآیند آن در اولین اجرا توسط خود دستور cargo run --release یا با اجرای دستور مستقل build cargo --release صورت می‌گیرد و دقایقی زمان خواهد برد).

۵ چالش‌های پیش‌آمده و راهکارهای حل آن‌ها

۱.۵ چالش اول: عدم دریافت پکت‌های UDP توسط سرور (مسدود بودن فایروال)

پس از راه‌اندازی، رابط کاربری با این که در وضعیت LIVE بود، پکت‌ها را دریافت نمی‌کرد و مقدار گره‌ها را 0 ESP32 Nodes نشان می‌داد. با بررسی متوجه شدیم فایروال اوبونتو (UFW) پورت ورودی 5005/UDP را مسدود کرده است.

راهکار: با اجرای دستور زیر، پورت باز شده و جریان دریافت داده‌ها بدون وقفه برقرار گردید:

```
sudo ufw allow 5005/udp
```

```
epo@hpds: ~  
epo@hpds:~$ sudo ufw allow 5005/udp  
Skipping adding existing rule  
Skipping adding existing rule (v6)  
epo@hpds:~$ _
```

شکل ۲: داشبورد وب پس از باز شدن پورت فایروال و اتصال موفقیت آمیز گره‌ها

۲.۵ چالش دوم: اختلاف زمانی دریافت پکت‌ها در سیستم چندنوده (Timestamp Spread)

با اضافه کردن گره دوم و سوم، سرور شروع به ادغام داده‌ها (Multi-Static Fusion) کرد اما به دلیل Clock Drift میکروکنترلرها و تاخیر متغیر شبکه (Network Jitter)، خطای زیر مداوم چاپ می‌شد:

```
WARN fusion error: Timestamp spread exceeds guard interval 60000 us
```

تحلیل و راهکار: طبق استانداردهای مخزن RuView، از مکانیزم تسهیم زمانی Time-Division Multiplexing (TDM) استفاده شد. با تنظیم متغیرهای اسلات زمانی، پنجره گارد تحمل تاخیر سرور برای ۳ نود افزایش یافت:

```
cd ~/Desktop/Amirali/WifiSensing/ruview/v2  
WDP_TDM_SLOTS=3 WDP_TDM_SLOT_US=200000 cargo run --release --package wifi-  
→ densepose-sensing-server -- \  
--udp-bind 0.0.0.0 \  
--udp-insecure-lan \  
--source esp32
```

۳.۵ چالش سوم: عبور امواج از دیوار و خطای شناسایی افراد خارج از اتاق

به دلیل خاصیت نفوذ امواج ۴.۲ گیگاهرتز از موانع گچی و شیشه‌ای (Through-Wall Propagation)، تردد افراد در راهرو مجاور نیز باعث نوسان فاز و افزایش خطای شمارش افراد می‌شد.

راهکار: برای حذف نویزهای ناشی از عبور از دیوار و متمرکز کردن ناحیه شناسایی بر داخل خود اتاق، آستانه حساسیت به تحرک (Motion Threshold) از طریق سرور بالاتر برده شد:

```
WDP_TDM_SLOTS=3 WDP_TDM_SLOT_US=200000 WDP_MOTION_THRESHOLD=1.5 cargo run --  
→ release --package wifi-densepose-sensing-server -- \  
--udp-bind 0.0.0.0 \  
--udp-insecure-lan \  
--source esp32
```

با اعمال این ضریب، سیگال‌های تضعیف شده ناشی از انعکاس دیوارهای خارجی فیلتر شده و شمارش و حضور پایدار گردید.

۶ نتیجه‌گیری

در نهایت، با پیکربندی دقیق ۳ گره ESP32-S3، تنظیم دسترسی‌های لایه شبکه، همگام‌سازی زمانی پکت‌ها با اسلات‌های TDM و بهینه‌سازی آستانه نویز، سیستم مش چندنوده RuView با موفقیت به صورت بلادرنگ راه‌اندازی و تثبیت گردید.

۷ روش نوین پیکربندی گره‌ها با استفاده از اسکریپت Provision.py

پس از آشنایی با روش سنتی پیکربندی از طریق menuconfig و کامپایل دستی، در ادامه به سراغ روش کارآمدتر و سریع‌تر رفتیم که امکان تغییر تنظیمات گره‌ها را بدون نیاز به کامپایل مجدد فراهم می‌کند. این روش مبتنی بر نوشتن مستقیم تنظیمات در پارتیشن NVS (Non-Volatile Storage) گره‌های ESP32-S3 است و مزیت‌های قابل‌توجهی از جمله سرعت بالا و عدم نیاز به کامپایل مجدد فریم‌ورک را به همراه دارد.

۱.۰۷ مراحل گام‌به‌گام پیکربندی با اسکریپت Provision.py

۱.۱.۰۷ گام اول: راه‌اندازی محیط و اتصال گره

ابتدا محیط توسعه ESP-IDF را فعال کرده و یک گره ESP32-S3 را از طریق کابل USB به سیستم متصل می‌کنیم. برای اطمینان از شناسایی پورت سریال، دستور زیر را اجرا می‌کنیم:

```
. $HOME/esp/esp-idf/export.sh
ls /dev/ttyACM*
```

پس از شناسایی پورت (به عنوان مثال /dev/ttyACM0)، دسترسی سریال را با دستور زیر فعال می‌کنیم:

```
sudo chmod 666 /dev/ttyACM0
```

۲.۱.۰۷ گام دوم: تنظیم معماری و پاک‌سازی حافظه فلش

در این مرحله، هدف معماری (Target) را روی esp32s3 تنظیم کرده و برای جلوگیری از تداخل فریم‌ورک‌های قبلی، حافظه فلش را پاک می‌کنیم:

```
cd ~/Desktop/Amirali/WifiSensing/ruview/firmware/esp32-csi-node
idf.py set-target esp32s3
idf.py -p /dev/ttyACM0 erase-flash
```

۳.۱.۰۷ گام سوم: فلش فریم‌ورک اصلی

پس از پاک‌سازی، فریم‌ورک اصلی پروژه را روی گره فلش می‌کنیم. توجه داشته باشید که این فریم‌ورک شامل قابلیت‌های جدید ارسال بسته‌های همگام‌سازی (Sync Packets) و پشتیبانی از ESP-NOW برای هماهنگ‌سازی زمانی است که در نسخه v0.8.4 پیاده‌سازی شده است:

```
idf.py -p /dev/ttyACM0 flash
```

پس از اتمام فلش، گره را با فشار دادن دکمه RST ریستارت می‌کنیم.

۴.۱.۷ گام چهارم: پیکربندی تنظیمات با اسکریپت Provision.py

در این مرحله، به جای استفاده از menuconfig و کامپایل مجدد، از اسکریپت provision.py برای نوشتن مستقیم تنظیمات در پارتیشن NVS استفاده می‌کنیم. این اسکریپت اطلاعاتی مانند SSID، رمز عبور، آی پی سرور، شناسه گره و تنظیمات TDM را روی گره ذخیره می‌کند. برای اولین گره (به عنوان مثال گره شماره ۱)، دستور زیر را اجرا می‌کنیم:

```
python provision.py --port /dev/ttyACM0
--ssid "Sharif-WiFi"
--password ""
--target-ip 172.27.2.87
--node-id 1
--tdm-slot 0
--tdm-total 3
```

پس از اجرای دستور، خروجی مشابه زیر مشاهده می‌شود:

```
Building NVS configuration:
WiFi SSID: Sharif-WiFi
WiFi Password: (empty)
Target IP: 172.27.2.87
Target Port: 5005
Node ID: 1
TDM Slot: 0 of 3
Flashing NVS partition (24576 bytes) to /dev/ttyACM0...
NVS provisioning complete!
```

این فرآیند در کمتر از یک ثانیه انجام می‌شود و نیازی به کامپایل مجدد نیست.

۲.۷ پیکربندی گره‌های بعدی در شبکه مش

برای افزودن گره‌های بعدی به شبکه مش، مراحل بالا را تکرار می‌کنیم با این تفاوت که پارامترهای node-id و tdm-slot را برای هر گره به صورت منحصر به فرد تنظیم می‌کنیم.

۱.۲.۷ گره شماره ۲ (اسلات ۱)

پس از اتصال گره دوم (که ممکن است روی پورت /dev/ttyACM1 شناسایی شود)، دستور زیر را اجرا می‌کنیم:

```
python provision.py --port /dev/ttyACM1
--ssid "Sharif-WiFi"
--password ""
--target-ip 172.27.2.87
--node-id 2
--tdm-slot 1
--tdm-total 3
```

۲.۲.۷ گره شماره ۳ (اسلات ۲)

به همین ترتیب برای گره سوم:

```
python provision.py --port /dev/ttyACM2
--ssid "Sharif-WiFi"
--password ""
--target-ip 172.27.2.87
--node-id 3
--tdm-slot 2
--tdm-total 3
```

۳.۷ مزایای روش Provision.py نسبت به روش سنتی

- سرعت بالا: پیکربندی هر گره در کمتر از یک ثانیه انجام می شود.
- عدم نیاز به کامپایل مجدد: تغییر تنظیمات بدون نیاز به کامپایل مجدد فریم ورک امکان پذیر است.
- قابلیت تغییر پویا: امکان تغییر تنظیمات شبکه و مش در هر زمان بدون نیاز به فلش مجدد.
- یکپارچگی با نسخه های جدید فریم ورک: پشتیبانی کامل از قابلیت های هماهنگ سازی زمانی در نسخه ۰.۸.۴v.

۴.۷ نکته مهم در خصوص شناسایی پورت گره ها

در صورتی که چند گره به سیستم متصل باشند، برای شناسایی پورت هر گره از دستور زیر استفاده می کنیم:

```
ls /dev/ttyACM*
```

خروجی این دستور لیستی از پورت های فعال را نمایش می دهد که می توانیم با اتصال و قطع هر گره، پورت متناظر با آن را شناسایی کنیم.

۵.۷ اجرای سرور با تنظیمات بهینه

پس از پیکربندی تمام گره ها، سرور را با تنظیمات بهینه برای شبکه مش ۳ گره ای اجرا می کنیم:

```
cd ~/Desktop/Amirali/WifiSensing/ruview/v2
WDP_GUARD_INTERVAL_US=150000 cargo run --release --package wifi-densepose-
  ↳ sensing-server --
--source esp32
--udp-port 5005
--udp-bind 0.0.0.0
--udp-insecure-lan
--http-port 3000
--ws-port 3001
--ui-path ../ui
```

با این روش، سیستم مش چندگره ای با موفقیت راه اندازی شده و داده های CSI از هر سه گره به صورت هماهنگ دریافت و پردازش می شوند.

آ پیوست‌ها و دستورات کمکی شبکه و سخت‌افزار

۱.آ شناسایی پورت سریال گره متصل شده (Port Detection)

برای تشخیص نام پورت اتصال گره‌های ESP32-S3 به سیستم اوبونتو، می‌توان از دستورات زیر استفاده کرد:

روش اول (بررسی سریع پورت‌های مدرن): بررسی میکند کدام پورت‌ها اشغال هستند.

```
ls /dev/ttyACM*
```

روش دوم (مانیتورینگ رویدادهای زنده هسته لینوکس): بلافاصله پس از اتصال کابل USB، دستور زیر پورت شناسایی شده را چاپ می‌کند:

```
dmesg | grep -i "tty" | tail -n 5
```

۲.آ استخراج آدرس IP کامپیوتر مقصد (Aggregator IP)

برای پیدا کردن آدرس IP محلی لپ‌تاپ جهت تنظیم در منوی menuconfig گره‌ها:

```
hostname -I | awk '{print $1}'
```

۳.آ تغییر IP پس از ری‌استارت و راه‌حل تثبیت آن (Static IP)

پرسش: در صورت اتصال با کابل LAN و خاموش/روشن شدن سیستم، آیا IP تغییر می‌کند و نیاز به پروگرام مجدد بردهاست؟

پاسخ و تحلیل: بله؛ در صورتی که کارت شبکه روی حالت DHCP (دریافت خودکار IP) تنظیم شده باشد، روتر شبکه ممکن است پس از ری‌استارت یک IP جدید اختصاص دهد. در این حالت پکت‌های ارسالی بردها به مقصد قبلی می‌روند و سرور چیزی دریافت نمی‌کند.

پیشنهاد: در صورت روشن و خاموش شدن مجدد کامپیوتر ممکن است نیاز به پروگرام کردن مجدد esp32 ها با IP جدید است.